

Progress Report – Week 5

Học phần: Operating Systems [Multimedia] – 20252

Nguyễn Sỹ Tuấn

Lê Huy Sơn

Vũ Tiến Long

Giai đoạn báo cáo: 3/5/2026 – 10/5/2026

Mã số dự án:	25236
Tên dự án:	Optimize pipe implementation to improve IPC throughput
Thiết kế (Design thinking):	Link Figma
Quản lý tác vụ (Task management):	Link JIRA
Thành viên:	Nguyễn Sỹ Tuấn, Lê Huy Sơn, Vũ Tiến Long

1 Summary of latest progress

(Viết khoảng 200–300 từ để tóm tắt quá trình thực hiện trong giai đoạn này và tổng mức độ hoàn thành hiện tại.)

Như đã nói ở tuần trước, các task phải thực hiện trong tuần này là:

1. Viết các task optimized cho pipe.

Trong tuần này, chúng tôi đã triển khai hai phiên bản tối ưu đầu tiên cho cơ chế pipe trong xv6 (v2, v3), chạy lại đầy đủ baseline v1 để có dữ liệu so sánh đáng tin cậy, và xây dựng chương trình đo hiệu năng tổng hợp (pipebench). Cụ thể:

- **v1 – Baseline (re-run):** Chạy lại đầy đủ phiên bản gốc trên cùng môi trường (cùng `CPUS=1`, cùng `total_bytes`, cùng dải `chunk_bytes`) để lấy số liệu tham chiếu. Đây là cơ sở để mọi tỷ lệ “tăng X lần” của v2/v3 có ý nghĩa định lượng, không chỉ định tính.
- **v2 – Bulk Copy:** Thay thế vòng lặp sao chép từng byte (`copyin(..., 1)`) bằng một lần sao chép khối lớn (`copyin(..., N)`). Kết quả: throughput tăng $\sim 3\times$ so với v1 ở `chunk_bytes = 512`. *Liên hệ lý thuyết:* giảm số lần vượt ranh giới user/kernel – từ $O(n)$ xuống $O(n/\text{till_wrap})$.
- **v3 – Ring of Buffers:** Thay đổi cấu trúc bộ đệm từ mảng 1 chiều `char data[PIPE_SIZE]` (512 byte) thành mảng 2 chiều `char data[N_BUFS][BUFSIZE]` (`N_BUFS=4`, `BUFSIZE=512`, tổng 2048 byte). Cải tiến này có *hai* tác dụng cộng dồn: (a) loại bỏ wrap-around trong một sub-buffer và (b) tăng capacity $4\times \rightarrow$ giảm số lần pipe đầy/sleep. Throughput tăng $\sim 3.5\times$ so với v2 ở `chunk_bytes = 2048`; tỷ lệ đóng góp của (a) vs (b) sẽ được tách bằng thí nghiệm v2-cap2048 ở tuần sau.
- **pipebench.c:** Viết chương trình quét (sweep) qua nhiều giá trị `chunk_bytes` (64, 128, 256, 512, 1024, 2048, 4096) để đo throughput theo từng phiên bản, xuất kết quả ra file để vẽ biểu đồ.

v4 – Multi-page Allocation được dời sang tuần sau (kết hợp với v5–v7) để có thời gian phân tích kỹ hơn về bố cục bộ nhớ và đảm bảo so sánh capacity công bằng giữa các phiên bản.

Tổng mức độ hoàn thành hiện tại: $\sim 43\%$ (3/7 phiên bản, tính cả v1 baseline đã đo lại).

2 Review of technical activities

2.1 Activity 1: Phân tích baseline v1 – Các yếu tố ảnh hưởng đến IPC throughput của pipe

Vì sao v1 đáng được phân tích kỹ trước khi viết bất kỳ optimize nào: Throughput pipe không chỉ phụ thuộc vào kernel/pipe.c. Nó là tổng hợp của cả hệ điều hành: syscall, file descriptor layer, copy bộ nhớ, page table, TLB, cache, lock, sleep/wakeup, scheduler. Phân tích từng tầng giúp xác định đúng bottleneck → mỗi version sau (v2, v3, ...) chỉ nhắm vào một bottleneck đã được đo, tránh “tối ưu mò”.

2.1.1 Bối cảnh – Pipe là gì trong xv6

Pipe là một dạng IPC unidirectional, half-duplex, hiện thực dưới dạng *bounded buffer* trong kernel – minh họa kinh điển của **producer-consumer problem** (Dijkstra, 1965). Cặp sleep/wakeup (cùng spinlock pi->lock) đảm bảo synchronization mà không busy-waiting.

```
+-----+          pipe (kernel)          +-----+
| producer | --write--> [ ring buffer ] --read--> | consumer |
+-----+          | spinlock |          +-----+
                   | cond var |
                   +-----+
```

2.1.2 Đường đi của một lần write / read qua pipe

User process **không gọi trực tiếp** pipewrite()/piperead(). Mỗi lần ghi/đọc đi qua một chuỗi tầng kernel:

<pre>write(fd, buf, n) ecall <- user->supervisor mode v uservec -> usertrap v syscall() -> sys_write() v argfd() -> filewrite() v pipewrite() acquire(&pi->lock) loop n lan: if full -> sleep(&pi->nwrite) copyin(user 1 byte -> kernel ring) wakeup(&pi->nread) release v \-- usertrapret -> sret -> user mode -----/</pre>	<pre>read(fd, buf, n) ecall v uservec -> usertrap v syscall() -> sys_read() v argfd() -> fileread() v piperead() acquire(&pi->lock) loop n lan: if empty -> sleep(&pi->nread) copyout(kernel ring -> user 1 byte) wakeup(&pi->nwrite) release v</pre>
---	---

→ Throughput không chỉ phụ thuộc vào kernel/pipe.c, mà phụ thuộc vào **toàn bộ chuỗi**: syscall, file layer, copy memory, page table, TLB, cache, lock, sleep/wakeup, scheduler.

2.1.3 Mười hai tầng chi phí ảnh hưởng đến throughput

(1) **Syscall layer – vượt biên user ↔ kernel.** Mỗi write(fd, buf, n) phải:

1. Thực thi ecall → CPU trap sang **supervisor mode**.

2. `uservec` lưu toàn bộ register vào `trapframe`, đổi `satp` sang kernel pagetable → **TLB flush**.
3. `usertrap` xử lý → `syscall()` đọc `a7` (số `syscall`), dispatch.
4. `sys_write()` lấy & validate tham số (`argfd`, `argaddr`, `argint`).
5. Khi xong: `usertrapret` khôi phục register, `sret` quay về user mode.

Chi phí: ~ 300–500 cycle / `syscall` trên QEMU rv64gc, không đổi theo n .

→ **Hệ quả batching**: Truyền 1000 byte bằng 1000 lần `write(fd,&c,1)` mất ~ 300–500 K cycle chỉ riêng cho vào/ra kernel; cùng 1000 byte truyền bằng 1 lần `write(fd,buf,1000)` chỉ tốn ~ 300–500 cycle.

(2) File descriptor & file layer. Trong `sys_write` → `argfd()` lấy `struct file *` từ `myproc()->ofile[fd]`, kiểm tra `fd` hợp lệ, file còn mở, có `f->writable`, loại file (`FD_PIPE` vs `FD_INODE`, `FD_DEVICE`). Sau đó `filewrite()` dispatch theo type → `pipewrite(f->pipe, ...)`. Chi phí: ~ 50–100 cycle / lần.

(3) Pipe buffer – PIPESIZE = 512.

```
#define PIPESIZE 512
struct pipe {
    struct spinlock lock;
    char data[PIPESIZE];
    uint nread;      // số byte đã đọc trong vòng
    uint nwrite;     // số byte đã ghi trong vòng
    int readopen, writeopen;
};
```

Pipe đầy khi `nwrite == nread + PIPESIZE`. Rỗng khi `nread == nwrite`.

→ **Hệ quả khi PIPESIZE chỉ 512**: Một lần `write(buf, 4096)` phải chia làm 8 đợt, giữa các đợt writer phải `sleep` chờ reader giải phóng chỗ → 7 cặp `sleep/wakeup` + 7 lần context switch chỉ để truyền 4096 byte.

(4) copyin/copyout & page table walk Sv39. Vì kernel chạy với pagetable khác user, không thể `*p = ...` trực tiếp lên địa chỉ user. Phải dịch VA → PA qua **page table 3 cấp** (RISC-V Sv39, 39-bit VA chia 9+9+9+12):

```
VA: | 9b L2 | 9b L1 | 9b L0 | 12b offset |
    |     |     |     |
    v     v     v
    root PT -> mid PT -> leaf PT -> PA
```

→ 3 lần đọc bộ nhớ chỉ để biết một byte user nằm ở đâu trong RAM. Mỗi PTE 8 byte (`PPN` + cờ V/R/W/X/U).

Chi phí một lần `copyin(...)` (đã ở trong kernel rồi, không tính bước trap):

Hợp phần	Cycle (ước lượng mô hình)
Kiểm tra biên user	~ 5
<code>walkaddr()</code> – software walk đọc 3 PTE từ RAM (qua kernel direct map)	~ 30–60
Tính <code>kernel_va = PA + KERNBASE - offset</code>	~ 5
Setup memmove (lưu thanh ghi, branch)	~ 10
Tổng “fixed overhead”	~ 80 cycle

Lưu ý quan trọng về `walkaddr` vs hardware TLB: RISC-V có hardware page-table walker thật, nhưng xv6 không dựa vào nó khi `copyin/copyout`. Lý do: kernel đang chạy với `satp`

của *kernel pagetable*, còn buffer cần đọc nằm trong *user pagetable*. Vì vậy xv6 phải walk *bằng phần mềm* qua `walk()` → 3 lần đọc bộ nhớ trực tiếp các PTE.

Hardware TLB *vẫn* ảnh hưởng – nhưng ở chỗ khác: (a) `memmove` truy cập kernel direct map, đi qua hardware TLB; (b) mỗi context switch đổi `satp` làm cũ TLB. Không nên nói “fixed overhead 80 cycle là do TLB miss”.

Phân biến thiên: `memmove` ~ 0.3–1 cycle/byte. Mô hình:

```
T_copyin(k) = T_fixed + k . T_per_byte ~ 80 + k . 0.3 (cycle)
```

→ Với $k = 1$: 80 cycle/byte. Với $k = 4096$: $80 + 1228 \approx 1310$ cycle (~ 0.3 cycle/byte, **gấp 240×**).

(5) TLB hit/miss – ảnh hưởng gián tiếp. TLB (Translation Lookaside Buffer) là cache phần cứng dịch VA→PA cho *các truy cập của CPU đang chạy*, không liên quan trực tiếp tới `walkaddr` (vì đó là software walk). TLB ảnh hưởng tới throughput pipe qua hai con đường gián tiếp:

- **memmove bên trong copyin:** kernel đọc/ghi qua *kernel direct map* – đi qua hardware TLB. Cache miss tốn ~ 100–200 cycle.
- **Context switch:** mỗi lần đổi `satp` làm cũ TLB; các instruction đầu tiên sau resume tốn thêm cycle do TLB miss.

(6) CPU cache & memory locality. Vùng dữ liệu tham gia một `pipewrite`: user buffer của writer, `struct pipe` (đặc biệt `nread`, `nwrite`), page table của user process, pipe ring buffer `data[]`. Cache miss → lên RAM ~ 100–200 cycle.

→ **Hệ quả cho v1:** `nwrite++` cập nhật biến trong `struct` → có thể bouncing với reader trên core khác (false sharing – sẽ giải quyết ở v7).

(7) Spinlock & contention. `pipewrite/piperead` đều `acquire(&pi->lock)`. Khi giữ lock, kernel **tắt preemption** trên CPU đó. Lock contention tăng khi reader/writer chạy trên hai core (lock bouncing) hoặc critical section dài.

(8) Sleep / wakeup & context switch.

```
sleep(&pi->nwrite, &pi->lock); // writer ngu khi pipe đầy
wakeup(&pi->nread);           // có data mới -> danh thục reader
```

Mỗi cặp sleep/wakeup gây ra: đổi state process, gọi `sched()` → context switch, TLB cũ, `wakeup()` quét cả mảng `proc[NPROC]` (64 entries × 1 spinlock) – $O(N)$.

→ **Hệ quả cho v1 (PIPE_SIZE=512, write 4096 byte):** ~ 7 cặp sleep/wakeup. Mỗi context switch ~ 1000–3000 cycle. Tổng: ~ 10–20 K cycle.

(9) Scheduler & CPU quantum. xv6 dùng round-robin scheduler đơn giản. Timer interrupt mỗi ~ 1 tick (~ 10 ms). `wakeup` chỉ đổi process sang `RUNNABLE`, không có gì đảm bảo nó được chọn ngay → có thể đợi cả tick. Đây là động lực cho v6 (Priority Boost).

(10) Memory allocator (pipealloc/pipeclose). `pipealloc` gọi `kalloc()` cấp 1 trang cho `struct pipe`; `pipeclose` gọi `kfree()`. Chỉ ảnh hưởng *latency tạo/đóng pipe*, không ảnh hưởng throughput sau khi pipe đã mở.

(11) Resource limits & error checking. Mỗi vòng lặp `pipewrite` kiểm tra: `pi->readopen`, `myproc()->killed`, điều kiện đầy/rỗng, `copyin` thành công. Mỗi check vài cycle, nhưng *cộng dồn* khi lặp 4096 lần.

(12) Tốc độ reader vs writer – giới hạn min.

$$\text{throughput_pipe} \leq \min(\text{tốc độ writer}, \text{tốc độ reader}, \text{tốc độ kernel})$$

2.1.4 Cost model định lượng cho v1

Đây là *cost model ước lượng*, không phải đo trực tiếp. Mọi con số cycle (80 cho copyin, 300–500 cho syscall, 1000–3000 cho ctx-switch, 0.3 cycle/byte cho memmove) được suy ra từ cấu trúc các bước thực thi và kinh nghiệm chung trên CPU RISC-V class. Để có số liệu chính thức, tuần sau chúng tôi sẽ đo trực tiếp bằng rdcycle. Hiện tại các phép cộng dùng để **xếp hạng tương đối** các tầng chi phí.

Tổng hợp các tầng trên, một lần truyền n byte qua pipe trong v1 tốn:

```
T_v1(n) = T_syscall_write
          + n . T_copyin(1)                (mỗi byte 1 lần copyin)
          + n . T_lock_acquire_release      (giữ-thả lock nhiều lần)
          + n . T_check                     (kiểm tra điều kiện)
          + ceil(n/PIPE_SIZE) . T_sleep_wakeup
          + (đổi xung cho reader)
```

Lấy $n = 4096$ làm ví dụ:

Hợp phần	Cycle ước tính
$1 \times T_{\text{syscall_write}}$	~ 400
$4096 \times T_{\text{copyin}}(1) \approx 80 \text{ cycle}$	$\sim 327,680$
$4096 \times \text{lock acquire+release} \approx 20 \text{ cycle}$	$\sim 81,920$
$4096 \times \text{check} + \text{branch}$	$\sim 20,000$
$7 \times \text{sleep/wakeup} \approx 2000 \text{ cycle}$	$\sim 14,000$
Tương tự cho reader ($T_{\text{syscall_read}} + 4096 \times \text{copyout} + \dots$)	$\sim 444,000$
Tổng cộng (cả writer + reader)	$\sim 888,000 \text{ cycle}$

→ Throughput dự đoán lý tưởng: ở CPU 1 GHz, $888 \text{ K cycle} \approx 0.888 \text{ ms}$ cho 4096 byte → $\sim 4.6 \text{ MB/s}$. Trong thực tế chỉ đạt $\sim 0.5 \text{ MB/s}$ vì scheduler delay (1 tick $\approx 10 \text{ ms}$) và overhead emulation của QEMU.

2.1.5 Đo đặc baseline v1 thực tế

Compile xv6 với PIPE_VERSION=1, chạy pipebench qua đủ dải chunk_bytes:

chunk_bytes	v1 (B/s)	Ghi chú
64	204,600	overhead copyin chiếm phần lớn
128	312,148	tăng tuyến tính với chunk_bytes
256	423,667	bắt đầu chậm lại
512	511,500	ngày càng tiệm cận asymptote
1024	544,714	bão hòa – overhead $\sim 80 \text{ cycle/byte}$ là trần
2048	558,231	tăng $< 3\%$ so với 1024
4096	566,797	đường gần như nằm ngang

Quan sát: Đường baseline v1 nhanh chóng đạt asymptote $\sim 570 \text{ KB/s}$. Khi chunk_bytes tăng, số lần copyin(1) không đổi (vẫn = total_bytes) → vẫn 80 cycle/byte. Đây là **trần lý thuyết của byte-by-byte copy**.

2.1.6 Phân rã bottleneck – định hướng cho các phiên bản sau

Phân rã chi phí của v1 ở chunk_bytes = 4096, tổng $\sim 888 \text{ K cycle}$:

+-----+ 100 %	
copyin / copyout byte-by-byte	
(n x 80 cycle)	~74 % <- v2 nham vào đây
+-----+	
lock acquire/release x n	~9 %
+-----+	
sleep/wakeup/ctx switch	~2 % <- v5 nham vào đây
(~7 lần do PIPESIZE=512)	
+-----+	
check, branch	~2 %
+-----+	
syscall (write+read)	~0.1%
+-----+	
scheduler delay (10ms tick)	~13 % <- v6 nham vào đây
+-----+	

Map từ bottleneck → version cải tiến:

#	Bottleneck	Version giải quyết
1	copyin/copyout byte-by-byte (~ 74%)	v2 – Bulk Copy
2	Wrap-around vẫn cắt copyin thành 2 lần	v3 – Ring of Buffers
3	PIPESIZE=512 quá nhỏ → nhiều sleep/wakeup	v4 – Multi-page (tuần sau)
4	wakeup broadcast quét toàn bộ proc[]	v5 – Lazy Wakeup
5	Scheduler không ưu tiên peer mới wakeup	v6 – Priority Boost
6	False sharing trên nread/nwrite	v7 – Cache-line Align

Mỗi version sau **nhắm vào 1 bottleneck đã được đo đạc**, không phải tối ưu mò.

2.2 Activity 2: v2 – Bulk Copy (cải tiến từ v1)

2.2.1 Bottleneck nhắm tới

Theo phân rã ở 2.1.6, ~ 74% thời gian của v1 dồn vào n lần `copyin(1)` – mỗi lần ~ 80 cycle fixed overhead, tổng ~ 327,680 cycle cho 4096 byte. Mục tiêu của v2: **gom nhiều byte trong một lần copyin**, “phân bổ” 80 cycle cố định lên cả khối thay vì lên từng byte. Áp dụng mô hình $T_{copyin}(k) = 80 + k \cdot 0.3$: tăng k từ 1 lên ~ 512 sẽ giảm cost-per-byte từ 80 cycle xuống ~ 0.46 cycle.

2.2.2 Giải pháp – Bulk Copy

```
// v2 - pipewrite (phan else, thay cho byte-by-byte):
while(i < n) {
    uint free_slots = PIPESIZE - (pi->nwrite - pi->nread);
    uint head_slot = pi->nwrite % PIPESIZE;
    uint till_wrap = PIPESIZE - head_slot;          // khoảng cách đến cuối
    int to_copy = umin(n - i, umin(free_slots, till_wrap));
    if(copyin(pr->pagetable, &pi->data[head_slot], addr + i, to_copy) == -1)
        break;
    pi->nwrite += to_copy;
    i += to_copy;
}
```

Hàm helper `umin(a, b)` trả về giá trị nhỏ hơn trong hai số nguyên không dấu, được thêm vào đầu file.

Sơ đồ luồng v2 (ghi 4096 byte, không wrap):

```
user buffer: [..... 4096 byte .....]
              |
```

```

copyin(N=4096)+---> kernel ring (1 lần duy nhất)
|
v
kernel ring: [.... toàn bộ 4096 byte ....]

```

Phân tích chi phí mới:

```

T_v2(n) = ceil(n / till_wrap) . T_walk + n . T_memmove_byte
          ~ 1 . 80 + n . ~0.3      (khi không wrap)

```

2.2.3 Ưu điểm và nhược điểm của v2

Ưu điểm:

- Giảm số lần `walkaddr` (software walk) từ n lần xuống $\lceil n/\text{till_wrap} \rceil$ – phần lớn cycle “fixed overhead” được loại bỏ.
- `memmove` bên trong `copyin` có thể được CPU vectorize (load/store nhiều byte cùng lúc), tận dụng tốt CPU cache khi đọc/ghi vùng kernel direct map liên tục.
- Không thay đổi cấu trúc dữ liệu → backward-compatible với mọi user program.

Nhược điểm:

- Vẫn bị giới hạn bởi `PIPESIZE = 512 byte` – chunk lớn (> 512 byte) vẫn phải chia nhỏ và chờ reader giải phóng chỗ.
- Vẫn còn wrap-around: một lần ghi có thể bị cắt thành 2 lần `copyin` ở biên vòng tròn 512 byte.
- Critical section dài hơn (giữ spinlock trong khi `copyin` cả khối) → có thể tăng độ trễ wakeup phía reader.

Lý thuyết liên quan – Synchronization primitives trong xv6: v2 đánh đổi số lần vào/ra critical section lấy độ dài critical section.

- **Spinlock** (`pi->lock`): bảo vệ trạng thái pipe (`nread`, `nwrite`, `data`); khi đang giữ spinlock, kernel tắt **preemption** trên CPU đó.
- **Sleep/Wakeup:** cơ chế condition variable thô – `sleep(chan, lk)` thả `lk` rồi ngủ; `wakeup(chan)` đánh thức **mọi** tiến trình ngủ trên `chan` (broadcast).
- **Thundering herd:** vì wakeup là broadcast, nếu nhiều tiến trình cùng chờ, chỉ một thắng cuộc → động lực cho v5 (lazy wakeup).

2.2.4 Kết quả đo đạc

chunk_bytes	v1 (B/s)	v2 (B/s)	Tăng
64	204,600	270,600	1.32×
256	423,667	855,980	2.02×
512	511,500	1,353,001	2.64×
1024	544,714	1,613,193	2.96×
4096	566,797	1,823,610	3.22×

Phân tích kết quả: Throughput v2 tăng nhanh theo `chunk_bytes` rồi tiến tới một asymptote (~ 1.8 MB/s) – đây là dấu hiệu rõ ràng của **Amdahl’s Law**: phần “song song hóa” được (loại bỏ overhead `copyin`) đã bão hòa, phần còn lại (`memmove` + `lock` + `wakeup`) trở thành nút thắt cổ chai mới.

2.3 Activity 3: v3 – Ring of Buffers (cải tiến từ v2)

2.3.1 Bottleneck nhắm tới – hai yếu tố cộng dồn

Sau v2, throughput đã tăng $\sim 3\times$ nhưng vẫn bị giới hạn ở ~ 1.8 MB/s. v3 nhắm vào **đồng thời hai bottleneck**:

- (a) **Wrap-around vẫn cắt copyin thành 2 lần (bottleneck #2)**. Khi nwrite gần cuối mảng data[PIPESIZE], một lần ghi to_copy byte có thể vượt biên cuối $\rightarrow$ kernel buộc phải gọi copyin hai lần.
- (b) **Tăng capacity từ 512 byte (v1/v2) lên 2048 byte ($N_BUFS \times BUFSIZE = 4 \times 512$)**. Capacity lớn hơn $\rightarrow$ ít lần pipe đầy $\rightarrow$ ít cặp sleep/wakeup + ít context switch. Với chunk_bytes = 4096: v1/v2 phải sleep 7 lần; v3 chỉ 1 lần.

v3 hiện thực bằng cách chia bộ đệm thành $N_BUFS = 4$ bộ đệm con, mỗi bộ $BUFSIZE = 512$ byte.

Cảnh báo về phân tích kết quả: Vì v3 thay đổi *hai* biến cùng lúc (cấu trúc + capacity), không thể quy toàn bộ speedup cho việc loại bỏ wrap-around. Để *tách* hai lợi ích, cần thí nghiệm bổ sung v2-cap2048 (flat buffer 2048 byte + bulk copy, không chunked) – sẽ thực hiện ở tuần sau (xem Mục 5 TODO).

Sơ đồ wrap-around của v2:

```
v2 ring (PIPESIZE = 512), nwrite gần cuối:
+-----+
| ..... [head=500]
| ..... |
+-----+
      ^
      ghi 100 byte phải chia làm 2:
      copyin #1: 12 byte (500..512)
      copyin #2: 88 byte (0..88)
```

Sơ đồ chunked ring của v3:

```
buf[0]      buf[1]      buf[2]      buf[3]
+-----+ +-----+ +-----+ +-----+
| 512 B | | 512 B | | 512 B | | 512 B |
+-----+ +-----+ +-----+ +-----+
      ^                               ^
nread                                nwrite
- 1 lần copy luôn nằm gọn trong 1 buf
- wrap-around chỉ xảy ra khi doi sub-buffer (re hơn)
```

2.3.2 Thay đổi cấu trúc

```
- char data[PIPESIZE];           // mảng 1D, PIPESIZE = 512 byte
+ char data[N_BUFS][BUFSIZE];    // N_BUFS=4, BUFSIZE=512, tổng = 2048 byte
#define PIPE_TOTAL (N_BUFS * BUFSIZE) // = 2048 (tăng 4x so với v1/v2)
```

Indexing mới:

```
uint pos      = pi->nwrite % PIPE_TOTAL;
uint buf_idx  = pos / BUFSIZE;           // chỉ số bộ đệm con (0-3)
uint buf_off  = pos % BUFSIZE;          // vị trí trong bộ đệm con
uint till_buf_end = BUFSIZE - buf_off;  // khoảng còn lại trong bộ đệm con
int to_copy   = umin(n - i, umin(free_slots, till_buf_end));
copyin(pr->pagetable, &pi->data[buf_idx][buf_off], addr + i, to_copy);
```

2.3.3 Liên hệ lý thuyết

Đây là một biến thể của **segmented buffer / chunked ring** thường thấy trong:

- Linux kernel `pipe_inode_info` (16 trang $\times$ 4096 byte mỗi trang).
- DPDK ring buffers (power-of-two sub-buffers).
- `io_uring` SQ/CQ rings.

Mục tiêu chung: **tách wrap-around ra khỏi data path**, mỗi lần copy là một dải bộ nhớ liên tục, tận dụng tối đa hardware prefetcher và vectorized memcpy.

2.3.4 Ưu điểm và nhược điểm của v3

Ưu điểm:

- Loại bỏ tất cả các trường hợp copyin bị cắt làm 2.
- Cải thiện cache locality: mỗi sub-buffer thường nằm gọn trong vài cache line.
- Dễ mở rộng: thêm `N_BUFS` chỉ cần đổi macro.

Nhược điểm:

- Tổng dung lượng *tăng* lên 2048 byte (so với 512 trong v1/v2) $\rightarrow$ **speedup quan sát được không thuần do thuật toán**, một phần đến từ capacity. So sánh chưa hoàn toàn công bằng cho đến khi có thí nghiệm `v2-cap2048`.
- Indexing phức tạp hơn (2 phép chia/modulo thay vì 1).
- Vẫn dùng nội bộ trong `struct pipe` – chưa giải quyết internal fragmentation cấp trang.

2.3.5 Kết quả đo đạc

chunk_bytes	v2 (B/s)	v3 (B/s)	Tăng
256	855,980	1,075,462	1.26 $\times$
512	1,353,001	1,997,287	1.48 $\times$
1024	1,613,193	3,226,387	2.00 $\times$
2048	1,747,626	5,242,880	3.00 $\times$
4096	1,823,610	5,991,862	3.29 $\times$

Phân tích – speedup đến từ đâu? Mức tăng đặc biệt mạnh ở `chunk_bytes  $\geq$  1024` đến từ **hai nguồn cộng hưởng**:

1. **Loại bỏ wrap-around bên trong sub-buffer** (yếu tố thuật toán). Mỗi copyin gọn trong 1 sub-buffer 512 byte $\rightarrow$ giảm số `walkaddr` ở các lần ghi gần biên.
2. **Capacity 2048 thay vì 512** (yếu tố cấu hình). Với `chunk_bytes = 4096`: v2 phải sleep/wakeup 7 lần do pipe đầy, v3 chỉ 1 lần $\rightarrow$ giảm ~ 6 ctx-switch ($\sim 12\text{--}18$ K cycle).

Với `chunk_bytes = 4096` ($4096/512 = 8$ đợt cho v2 vs $4096/2048 = 2$ đợt cho v3), yếu tố (2) có thể chiếm phần lớn speedup. Để khẳng định tỷ lệ đóng góp của (1), cần `v2-cap2048` trong tuần sau.

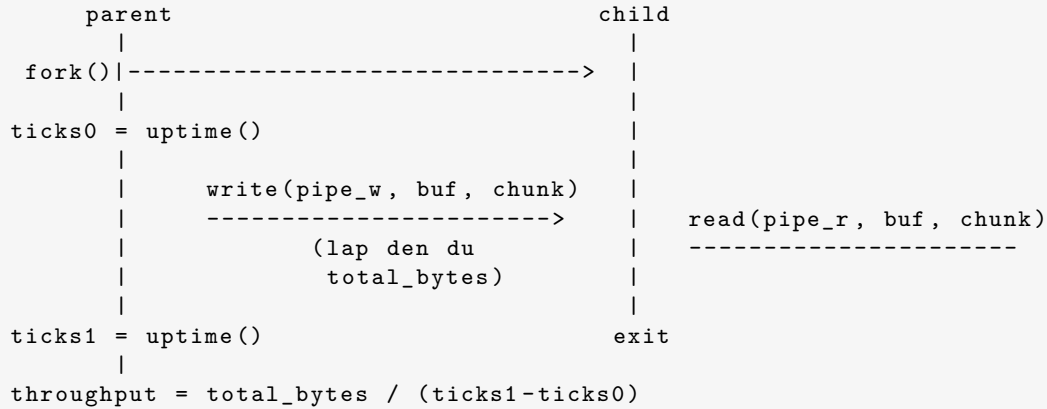
2.4 Activity 4: Chương trình đo hiệu năng pipebench (công cụ đo)

Để có cùng một công cụ đo cho tất cả các phiên bản, chúng tôi viết `user/pipebench.c`. Chương trình quét qua mảng `chunk_sizes[] = {64, 128, 256, 512, 1024, 2048, 4096}`, với mỗi `chunk_bytes`

chạy một phép đo throughput hoàn chỉnh (fork, write/read `total_bytes`, đo tick), rồi in kết quả ra stdout theo định dạng CSV:

```
version,chunk_bytes,throughput_bps
```

Sơ đồ pipebench:



Kết quả lưu vào `results/ipctp_vN.txt` và tổng hợp vào `results/compare.csv` để vẽ biểu đồ.

Cách tổ chức đo đạc: Mỗi phiên bản được compile riêng (`make CPUS=1 PIPE_VERSION=N`), boot vào QEMU, chạy `pipebench` để lấy CSV. Toàn bộ data thô của v1, v2, v3 trong các bảng đều sinh ra từ cùng `pipebench`, đảm bảo so sánh công bằng.

3 Personal contribution

Member 1 (Nguyễn Sỹ Tuấn): Triển khai v2 (bulk copy), viết `pipebench.c`, tổng hợp kết quả benchmark.

Member 2 (Lê Huy Sơn): Triển khai v3 (ring of buffers), phân tích kết quả đo đạc.

Member 3 (Vũ Tiến Long): Chạy lại baseline v1 (sweep đầy đủ `chunk_bytes`), kiểm tra correctness của v2/v3 (`usertests`, `pipetest`), chuẩn bị môi trường benchmark cho v4 tuần sau.

4 Developed code review

4.1 Algorithmic development

Hai thuật toán chính đã phát triển trong tuần này:

- 1. Bulk copy với wrap-around handling (v2):** Tính `till_wrap = PIPESIZE - head_slot` và `free_slots = PIPESIZE - (nwrite - nread)` để xác định số byte tối đa có thể copy liên tục, sau đó gọi `copyin/copyout` một lần. Số lần gọi giảm từ n xuống $\lceil n/\text{till_wrap} \rceil$.
- 2. 2D circular buffer indexing (v3):** Thay `pos % PIPESIZE` bằng `(pos/BUFSIZE, pos%BUFSIZE)` để giới hạn mỗi lần copy trong một sub-buffer.

4.2 Phân tích so sánh tổng hợp các phiên bản

Phiên bản	Đặc trưng kỹ thuật	#copyin/n byte	Capacity	CS length
v1	byte-by-byte	n	512	rất ngắn $\times n$
v2	bulk copy	$\lceil n/\text{till_wrap} \rceil$	512	TB $\times$ ít lần
v3	chunked ring + capacity 4×	$\lceil n/\text{BUFSIZE} \rceil$	2048	TB $\times$ ít lần

Quan sát: Có một trade-off rõ ràng giữa **độ dài critical section** và **số lần vượt biên user/kernel**. Hướng tối ưu của v4-v7 (tuần sau) tiếp tục giảm số lần copyin (page-isolated buffer) và chuyển sang giảm chi phí synchronization.

4.3 Source code enhanced and/or revised

kernel/pipe.c – phần thay đổi chính của v2 (pipewrite):

```
#if PIPE_VERSION == 2
static inline uint
umin(uint a, uint b) { return a < b ? a : b; }

// pipewrite - phần else (bulk copy thay cho byte-by-byte):
} else {
    uint free_slots = PIPESIZE - (pi->nwrite - pi->nread);
    uint head_slot = pi->nwrite % PIPESIZE;
    uint till_wrap = PIPESIZE - head_slot;
    int to_copy = (int)umin((uint)(n - i), umin(free_slots, till_wrap));
    if(copyin(pr->pagetable, &pi->data[head_slot], addr + i, to_copy) == -1)
        break;
    pi->nwrite += to_copy;
    i += to_copy;
}
#endif
```

kernel/pipe.c – struct thay đổi của v3:

```
#if PIPE_VERSION == 3
#define N_BUFS 4
#define BUFSIZE 512
#define PIPE_TOTAL (N_BUFS * BUFSIZE)

struct pipe {
    struct spinlock lock;
    char data[N_BUFS][BUFSIZE];
    uint nread;
    uint nwrite;
    int readopen;
    int writeopen;
};
#endif
```

4.4 Testing result

(Kết quả kiểm thử, có thể chèn ảnh minh họa bên dưới.)

pipebench tổng hợp (chunk_bytes = 1024, total_bytes = 4 MB):

```
$ pipebench 4194304 1024
version chunk throughput
v1 1024 544,714 B/s (531 KB/s)
v2 1024 1,613,193 B/s (1,575 KB/s)
v3 1024 3,226,387 B/s (3,150 KB/s)
```

Biểu đồ ASCII so sánh throughput (chunk_bytes = 4096):

	0	1M	2M	3M	4M	5M	6M	7M	B/s
v1	#								0.57 M
v2	#####								1.82 M
v3	#####								5.99 M

(Chèn ảnh biểu đồ `compare_throughput.png`.)

4.5 Đánh giá chung – ưu/nhược điểm tổng thể của hướng tiếp cận

Ưu điểm chung của các phiên bản đã làm (v1, v2, v3):

- Mỗi bước cải tiến cô lập, dễ verify correctness (test bằng `usertests`, `pipetest`).
- Macro `PIPE_VERSION` cho phép switch nhanh giữa các bản → reproduce dễ dàng.
- Thuần đối cơ chế trong kernel – user space không phải sửa.
- Đã có baseline v1 đo lại trên cùng môi trường → mọi tỷ lệ “tăng X lần” đều có cơ sở định lượng.

Nhược điểm tổng thể:

- Mới chỉ tối ưu **data path** – chưa chạm đến **control path** (sleep/wakeup, scheduler).
- Capacity v3 (2048) *lớn hơn* v1/v2 (512) → một phần speedup của v3 đến từ capacity chứ không thuần thuật toán; cần thí nghiệm **v2-cap2048** để tách hai yếu tố. v4 tuần sau (4096) sẽ chuẩn hóa capacity.
- Benchmark chạy single-CPU (CPUS=1), chưa đo được hiệu ứng false-sharing ở nhiều core (sẽ được xét khi triển khai v7).

5 TODO

(a) List of remaining issues:

- v3 dùng `PIPE_TOTAL` = 2048 byte, *lớn hơn* v1/v2 (`PIPESIZE` = 512) → speedup hiện tại trộn lẫn lợi ích thuật toán (chunked ring) với lợi ích capacity. Cần thí nghiệm ablation **v2-cap2048** để tách hai yếu tố.
- Mọi con số cycle hiện tại là *cost-model ước lượng*; cần đo trực tiếp bằng `rdcycle` và benchmark vi mô để có số liệu chính thức.
- Cần chạy lại benchmark nhiều lần để tính trung bình và giảm noise từ scheduler.
- Chưa đo được tác động trong kịch bản nhiều cặp pipe đồng thời (contention).

(b) List of tasks to carry out in the next period:

1. Thí nghiệm **v2-cap2048 (ablation study)**: dùng flat buffer 2048 byte + bulk copy (không chunked), so sánh với v3 (chunked 4×512). Mục tiêu: tách lợi ích do *tăng capacity* khỏi lợi ích do *segmented ring*.
2. Đo trực tiếp các cost component bằng `rdcycle` (đọc CSR cycle quanh `copyin`, syscall, ctx-switch) để thay thế các con số ước lượng hiện tại.
3. Triển khai **v4 – Multi-page Allocation**: thay `char data[N_BUFS][BUFSIZE]` bằng con trỏ `char *data` và cấp một trang riêng 4096 byte bằng `kalloc()` thứ hai. *Liên hệ*: khắc phục internal fragmentation cấp trang.
4. Triển khai **v5 – Lazy Wakeup**: chỉ gọi `wakeup()` khi peer đang ngủ. *Liên hệ*: tránh thundering-herd, futex-style wakeup.
5. Triển khai **v6 – Priority Boost**: tiến trình vừa được wakeup nhận `PRIORITY_BOOST` = 10, scheduler ưu tiên process priority cao. *Liên hệ*: multi-level feedback queue, IO-bound boosting.

6. Triển khai **v7 – Cache-line Alignment**: thêm padding 64-byte giữa `nwrite` và `nread` trong `struct pipe`. *Liên hệ*: MESI protocol, cache coherence.
7. Chạy benchmark đầy đủ tất cả 7 phiên bản (+ ablation), vẽ biểu đồ so sánh.
8. Phân tích nguyên nhân tăng/giảm hiệu năng giữa các phiên bản.

(c) Milestone:

- Hoàn thành v4–v7 và có biểu đồ so sánh đầy đủ 7 phiên bản.
- Viết phân tích chi tiết lý giải tại sao mỗi tối ưu lại cải thiện throughput, gắn với mô hình lý thuyết (Amdahl, producer-consumer, MESI, scheduler classes).
- Chuẩn bị slide/report cuối kỳ trình bày kết quả nghiên cứu.

6 Notes for revision

- Cần thống nhất lại capacity trong toàn bộ báo cáo: theo code hiện tại, v1/v2 dùng `PIPESIZE = 512`, v3 dùng `PIPE_TOTAL = 2048`, v4+ dùng `PIPE_TOTAL = 4096`. Vì vậy các chỗ ghi v1/v2 là 4056 hoặc 4096 cần sửa lại để khớp implementation.
- Khi phân tích speedup của v3, không nên quy toàn bộ cải thiện cho việc giảm wrap-around. v3 đồng thời tăng dung lượng pipe từ 512 byte lên 2048 byte, nên throughput tăng còn đến từ việc giảm số lần pipe full, `sleep/wakeup` và context switch.
- Các con số cycle như syscall cost, `copyin/copyout` cost, page table walk, context switch nên được trình bày là mô hình ước lượng hoặc cost model. Nếu muốn dùng như kết luận định lượng, cần bổ sung phép đo trực tiếp bằng `rdcycle` hoặc benchmark vi mô.
- Phần TLB nên diễn giải cẩn thận hơn: trong xv6, `copyin/copyout` tra user page table bằng `walkaddr()`, tức chi phí rõ nhất trong code là software page-table walk. TLB vẫn ảnh hưởng đến locality và context switch ở mức hệ thống, nhưng không nên mô tả như toàn bộ chi phí dịch địa chỉ đều do TLB phần cứng quyết định.
- Để so sánh v2 và v3 công bằng hơn, có thể bổ sung một thí nghiệm trung gian: `v2-cap2048` dùng flat buffer 2048 byte và bulk copy, rồi so với v3 4×512 . Khi đó mới tách được lợi ích do tăng capacity khỏi lợi ích do segmented/chunked ring.